

Reviewer Pack — Minimal Rank of Tropical Curves and Resolution Refutation Le...

Entry b2acbd351d58 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 12:25:25 UTC

Minimal Rank of Tropical Curves and Resolution Refutation Length

Entry ID: b2acbd351d58 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 12:25:25 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For every Tropical Curve T in the projective plane defined by a set of tropical polynomials, the length of any Resolution refutation for the corresponding Tseitin formula is at least $2^{\text{rank}(T)}$.’, ‘Here, $\text{rank}(T)$ denotes the maximum dimension of any irreducible component of the zero-locus of T .’, ‘For all instances with no irreducible components, $\text{rank}(T) = 0$.’]

Rationale (proposer’s reasoning):

[‘Tropical Geometry provides a rich structure for analyzing algebraic properties of computational problems. The conjecture leverages this structure to provide a possible lower bound on Resolution refutation length, potentially revealing the inherent complexity in certain Tseitin formulas.’, ‘The rank of a tropical curve captures the complexity of its underlying algebraic structure, which may correlate with the difficulty of finding proofs for associated Tseitin formulas.’, ‘If true, this conjecture would provide a new way to prove lower bounds on Resolution proof length using geometric tools.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f095d2192679b107

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all tropical curves T , the mean number of steps in any resolution refutation exceeds $2^{\text{rank}(T)}$ across at least 80% of the trials and does not exceed this value by more than 3 standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND resolution proof complexity AND minimal rank - Tseitin formula AND tropical curve rank AND Resolution refutation length - irreducible component AND zero-locus AND $2^{(\text{rank}(T))}$ in tropical geometry AND resolution proof complexity

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1306.3497v2] The number of vertices of a tropical curve is bounded by its area - [http://arxiv.org/abs/1001.1554v4] Tropical geometry and correspondence theorems via toric stacks - [http://arxiv.org/abs/math/0406099v5] A tropical calculation of the Welschinger invariants of real toric Del Pezzo surfaces - [http://arxiv.org/abs/1506.04846v1] Analytification and Tropicalization over non-archimedean fields - [http://arxiv.org/abs/1608.02306v2] Three dimensional tropical correspondence formula - [http://arxiv.org/abs/0902.4637v2] The p-rank strata of the moduli space of hyperelliptic curves - [http://arxiv.org/abs/2305.02692v1] Irreducible representations of the twisted Heisenberg-Virasoro algebra of Hom type

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random tropical curve of degree n
    n = random.randint(5, 40)
    variables = [f'x{i}' for i in range(n)]
    negated_literals = [f'~{var}' for var in variables]
    clause = random.sample(variables + negated_literals, random.randint(1, n))

    # Construct the Tseitin formula
    tseitin_formula = []
    literals = set()
    for lit in clause:
        if lit.startswith('~'):
            literals.add(lit[1:])
```

```

    else:
        literals.add(lit)

for var in literals:
    tseitin_formula.append([var, f'~{var}', 'OR'])
    tseitin_formula.append([f'~{var}', f'~{var}', 'AND'])
    tseitin_formula.append([f'{var}', f'~{var}', 'XOR'])

# Simulate a DPLL-based solver to find refutation length
def dpll(formula, assignment):
    if not formula:
        return 0
    literal = next(lit for lit in formula[0] if lit.startswith('~') != formula[1][lit])
    if literal in assignment and assignment[literal] != formula[1][literal]:
        return float('inf')
    new_assignment = assignment.copy()
    new_assignment[literal] = formula[1][literal]
    return 1 + min(dpll(formula[2:], new_assignment), dpll([f'~{lit}'] for lit in formula), new_assignment)

refutation_length = dpll(tseitin_formula, {})

# Compute the rank of the tropical curve (simplified example)
rank = len(variables) # This is a placeholder; actual computation depends on the curve's zero locus

return {
    "metric_name": "refutation_length",
    "metric_value": refutation_length,
    "instances_tested": 1,
    "conjecture_holds": refutation_length >= 2 ** rank,
    "counterexample": "" if refutation_length >= 2 ** rank else f"Refutation length {refutation_length} < 2^{rank}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_refutation_length = sum(result["metric_value"] for result in results) / len(results)
    std_refutation_length = math.sqrt(sum((result["metric_value"] - mean_refutation_length) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_refutation_length} std={std_refutation_length} support={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Refutation length < 2^{rank}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_223af37b.py", line 69, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_223af37b.py", line 51, in run_trial
    refutation_length = dpll(tseitin_formula, {})
                       ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_223af37b.py", line 44, in dpll
    literal = next(lit for lit in formula[0] if lit.startswith('-') != formula[1][lit])
               ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_223af37b.py", line 44, in <genexpr>
    literal = next(lit for lit in formula[0] if lit.startswith('-') != formula[1][lit])
               ~~~~~^
TypeError: list indices must be integers or slices, not str
```

8. Critic adversarial review

Critic verdict: CHALLENGE
Critic reasoning:
skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE
Reasoning:
The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the test with appropriate error handling and debugging to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15595
2	propose	ollama_remote	glm4:latest	0	0	14641
3	preregistration	ollama_remote	glm4:latest	0	0	9494
4	novelty	ollama_remote	glm4:latest	0	0	8554
5	novelty	ollama_remote	glm4:latest	0	0	9374
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17655
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7045
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17010
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9402
10	judge	ollama_remote	glm4:latest	0	0	11424

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120195 ms total latency. Provider mix: {'ollama_remote': 10}
(full prompt+response transcripts available in research/audit/b2acbd351d58.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b2acbd351d58.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b2acbd351d58.tar.gz` (if generated)